Table of contents:

2- BBC

3- Galala University

1- Amazon

URL: amazon.com

1.1. Summary

This section analyzes Amazon’s homepage performance using DevTools, Lighthouse, and Wappalyzer, the focus is on identifying the technologies used to build the website and evaluating its performance using key such as page load time, total page size, HTTP requests, and Core Web Vitals, the goal is to understand why Amazon performs well despite its large scale and heavy amount of content.

1.2. Methodology

This analysis has used the following tools in order to achieve the conducted results:

- Browser DevTools (Network tab) to measure page size, load time, image count, and HTTP requests.
- Lighthouse to gather Core Web Vitals such as LCP, INP, and CLS.
- Wappalyzer to detect Amazon’s technologies, CMS, frameworks, and libraries.

Each measurement was taken three times to avoid temporary network fluctuations, and the numbers were averaged from both mobile and desktop, they were tested to compare differences, and the goal was to determine how Amazon optimizes such a large homepage while maintaining high performance.

1.3. Technologies stack identification

1.3.1. Server location

Amazon uses multiple servers globally, with dynamic routing based on user location. For this report, the server used resolved to the **United States**. This improves latency for US users but also demonstrates Amazon's large-scale distributed architecture, which helps them deliver content quickly regardless of user region.

1.3.2. Content management system

Amazon uses their own custom written content management system (CMS) to ensure the best performance for their website; the name of the content management system (CMS) is called Amazon Web Services (AWS), which they also use as a cloud computing platform.

1.3.3. Frontend framework

Amazon relies on a custom frontend framework integrated with AWS Amplify. It also incorporates familiar libraries from the Amplify ecosystem such as React, Vue, and Angular, but wrapped in their own optimized

system. This hybrid approach allows Amazon to load only the necessary components, improving speed while still supporting modular UI development.

1.3.4. Javascript libraries

jQuery is one of the JavaScript that Amazon uses for their website, alongside that they also use Core-JS and Lodash, but mainly jQuery due to its efficiency, small and rich libraries, making it one of the most used JavaScript libraries.

1.4. Initial Measurements

Page load time	1-3 seconds
Total page size	6.81 Mbs
HTTP Requests	199-376
Number of images	153-314

1.4.1. Page load time

Amazon’s page load time averaged **2–3 seconds**, which is impressive considering the hundreds of images and dynamic content. The fast loading time is primarily due to aggressive image compression, optimized CDN distribution, and lazy loading for non-critical elements. Amazon prioritizes

visible content, allowing users to interact with the website before everything finishes downloading.

1.4.2. Total page size

The page size of 6.81 MB is larger than the recommended size, mostly because of product images, promotional banners, and dynamic content, despite being relatively heavy the website handles this efficiently using optimization strategies such as WebP images, image compression, and deferred loading for non-critical elements, these methods allow the main content to appear quickly while other assets load in the background, as a result the large page size does not noticeably slow down the user experience, overall Amazon manages to keep the website fast and responsive even with a heavy page.

1.4.3. Number of HTTP requests

Amazon triggers 200–376 HTTP requests, with images making up most of them, a high number of requests would normally slow down a website, but Amazon reduces this impact by loading assets asynchronously and using caching to prevent repeated downloads, many requests are only triggered when elements scroll into view, this approach ensures that the initial page loads quickly while additional content loads as needed, overall the website handles a large number of requests efficiently without affecting user experience or responsiveness

1.4.4. Number of images

From the previous point, the number of images used vary a lot on amazon, but they can go from 153-314 images alone, making most of the HTTP requests, that is due to amazon containing so many different images for their products on the homepage, it’s variation depends on the user’s search history and what they looked up before since amazon shows different images depending on that information.

1.5. Core Web Vitals

Measurement	Mobile	Desktop
LCP	1.4s	1.4s
INP	168ms	114ms
CLS	0.0	0.1

1.5.1. Mobile analysis

1.5.1.1. LCP

The Largest Contentful Paint (LCP) for Amazon on mobile is 1.4 seconds, which is well below the recommended 2.5 second threshold set as the standard for good performance, the result is impressive because Amazon’s homepage contains a large number of images, banners, and product previews that normally slow down loading time, the website manages to keep the LCP low by compressing images, using efficient formats, and loading only what is needed at the start, Amazon also uses strong caching and a global content delivery network (CDN), which helps deliver content faster to mobile devices, this level of optimization ensures that the main

visible content appears quickly, providing a smooth first impression for users.

1.5.1.2. INP

The Interaction to Next Paint (INP) on mobile measures around 168ms, which is below the recommended limit of 200ms for good responsiveness. This means that user actions such as tapping buttons, scrolling, or opening menus register very quickly without noticeable delay. After testing it myself on a mobile phone, the website consistently responded immediately to interactions, showing no lag or slowdowns even when browsing different product categories. Amazon achieves this by minimizing heavy JavaScript operations, optimizing event handling, and ensuring that interactive components are not blocked by other loading processes. Overall, this leads to a very responsive browsing experience on mobile devices.

1.5.1.3. CLS

Amazon’s mobile version has a Cumulative Layout Shift (CLS) score of 0, meaning there are no unexpected layout movements while the page loads. This is important because layout shifting can frustrate users, especially if they are about to tap something and the content moves suddenly. During testing on my own phone, the layout stayed completely stable as images, text, and links loaded, confirming the CLS value. Amazon prevents layout shifts by reserving proper space for images, loading fonts correctly, and ensuring banners and ads do not suddenly jump into place. A CLS score of 0 gives a very consistent and comfortable reading and browsing experience for mobile users.

1.5.2. Desktop analysis

1.5.2.1. LCP

The largest contentful paint (LCP) for Amazon on desktop is also 1.4 seconds, which is below the target of 2.5 seconds, similar to the mobile version, the desktop site achieves this speed through several optimizations including image compression, lazy loading of non-critical elements, and the use of a global content delivery network, these strategies ensure that even with many images, banners, and interactive components the main content appears quickly, making the desktop experience fast and smooth for users, overall the LCP shows that Amazon maintains strong performance across all devices.

1.5.2.2. INP

The time for interaction to next paint (INP) is around 114ms, which is again below the target of 200ms, testing it myself on my desktop shows that the website responds instantly to clicks and shows no signs of delay, every interaction feels smooth and consistent, it is also clear that the desktop version performs slightly faster than the mobile version, this is expected because desktop devices usually have stronger hardware and can process interactions more quickly, overall the INP score shows that Amazon delivers a very responsive experience on desktop.

1.5.2.3. CLS

Amazon has 0.1 Cumulative Layout Shift (CLS) on desktop, the website shows a very small amount of layout shifting, testing it on my desktop I can barely notice any movement, the slight layout shift happens because desktop screens are larger and contain more elements that need to load,

with more images and sections loading at the same time there is a tiny shift as everything settles, the shift is minimal and does not affect the user experience.

1.5.3. Comparison

The differences between mobile and desktop are very minimal, desktop appears to be slightly faster than mobile by around 40ms, this small difference is expected since desktop devices usually have stronger hardware and can process larger pages more quickly, however desktop also has more layout shifting compared to mobile, this happens because the desktop layout is bigger and wider, it contains more elements, more sections, and more moving parts, all of these increase the chance of small shifts during loading, despite this the overall experience on both platforms remains smooth and responsive with only minor variations between them.

1.6. Lighthouse findings

1-Render blocking requests

Render blocking requests delay the browser from showing the first visible content, because these files must load fully before the page can begin rendering anything, that means the browser is forced to wait for CSS and JavaScript files to download, parse, and execute before it can display the layout, As a result, the start of the page appears slower to users, especially on weaker devices or slower networks, when too many render blocking files exist, the page remains blank longer, which directly affects loading speed and responsiveness, that also causes the Largest Contentful Paint (LCP) to take longer to appear, since the main content cannot load until these files finish. Reducing these requests makes the page feel faster and improves overall user experience.

2- Font display

Without a proper font-display setting, text may remain invisible while waiting for the custom font to load, causing a delay that makes the site feel broken or empty. This behavior is called a flash of invisible text and is common on slower connections. By setting font-display to swap or optional, the browser shows fallback text immediately, allowing users to read content without waiting. The custom font then loads in the background and replaces the fallback once ready. Using swap can sometimes cause layout shifts, but this can be improved by adjusting font metrics or line height to match the fallback font. Proper font-display settings make text appear faster and improve stability and readability across devices.

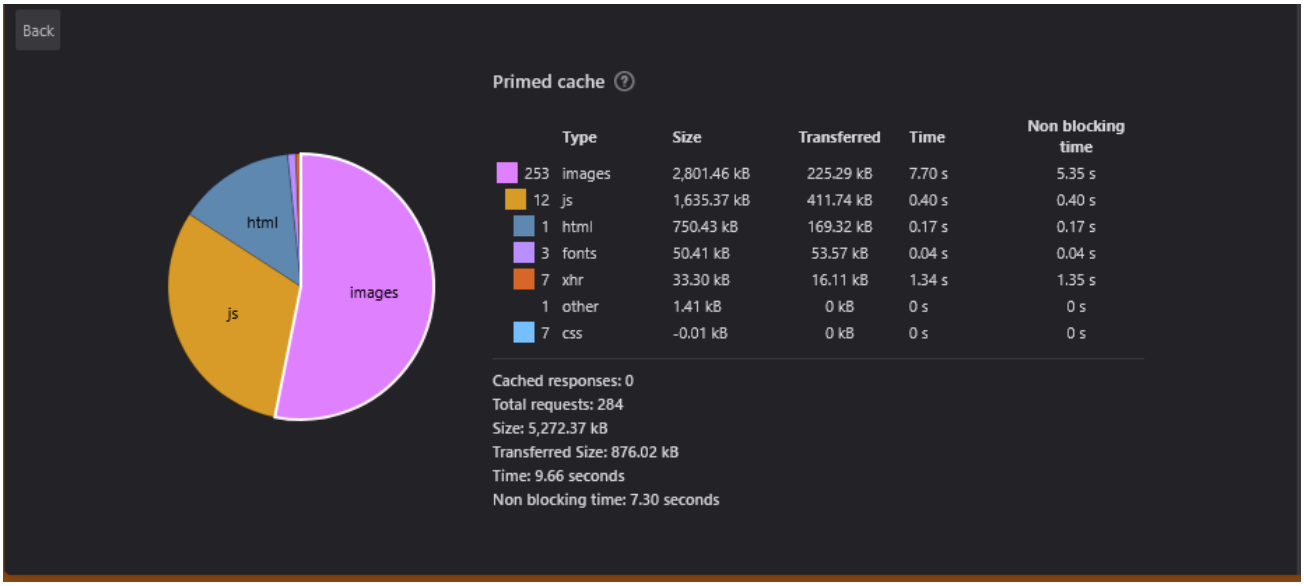
3- Legacy JavaScript

Using old or outdated JavaScript versions can cause performance issues because modern browsers are optimized for newer standards and features, legacy JavaScript often contains larger code bundles, slower execution, and less efficient patterns compared to modern alternatives, because of this, this increases load times and slows down the overall page performance, especially for interactive elements, a lot of modern tools, libraries, and frameworks expect up-to-date JavaScript, so relying on older versions can also lead to compatibility problems, as well as, Legacy code may also lack security updates, which introduces risks over time, so the solution is updating JavaScript, it will improves speed, reduces file size, and ensures the website can use modern browser optimizations effectively.

1.7. Images and data used in the analysis

This section will contain all the information and data used in this analysis, as well as how I was able to get those images and data, I will be using this data to validate and proof all the information I have presented in this analysis

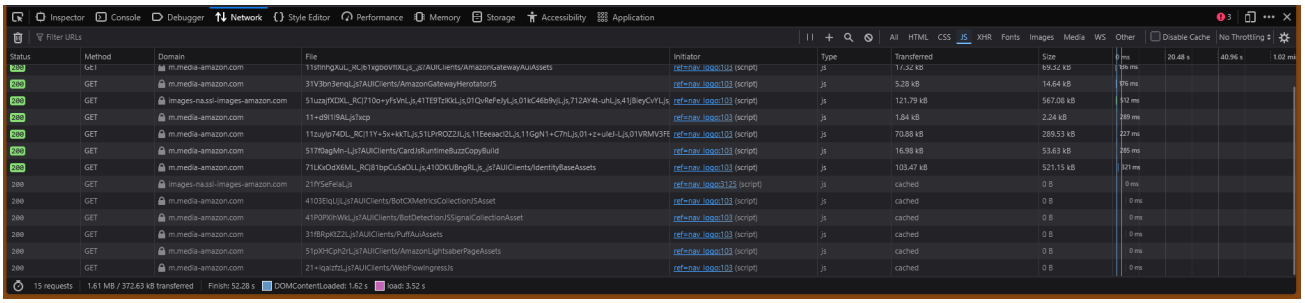
1.7.1. Network analysis



This image was used to collect information about the number of images and number of HTTP requests done in the website, it was obtained by opening the Amazon website, clicking inspect website, going to the network section and then pressing network analysis (it was a timer button), this was done on the firefox website.

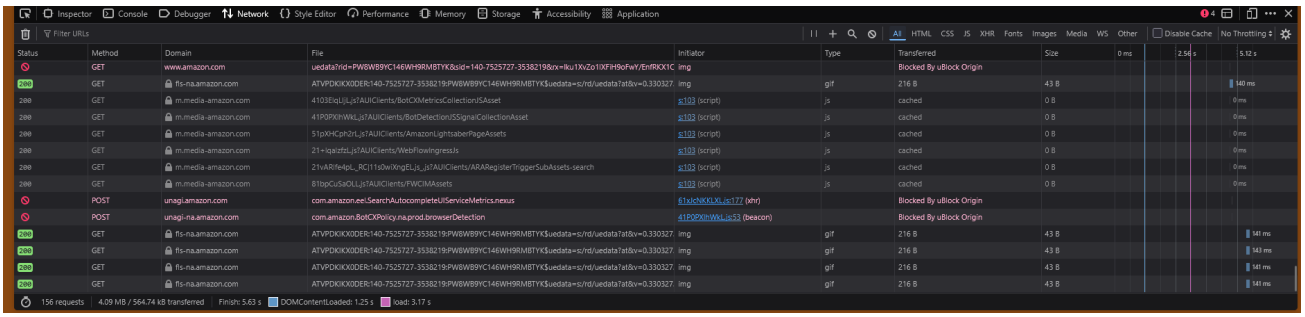
1.7.2. Images used for analysis method

1.7.2.1.



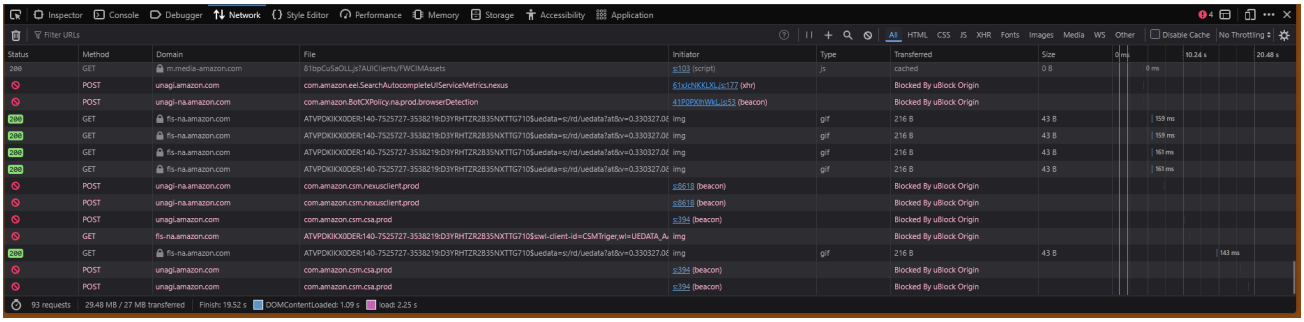
This image is the first out of the 3 pictures used to collect loading time, it was done early in the day when the server had the most traffic, thus showing the slowest loading time by 3.52 seconds, compared to the other pictures taken through the day that have less time thanks to the less traffic.

1.7.2.2.



This image was captured a few hours after the first image, it shows a slightly faster load time which shows that the traffic is less compared to earlier in the day, this shows that stuff like traffic that’s outside the website’s owner ability to optimize can’t be predicted and will cause issues if not taken any precautions for.

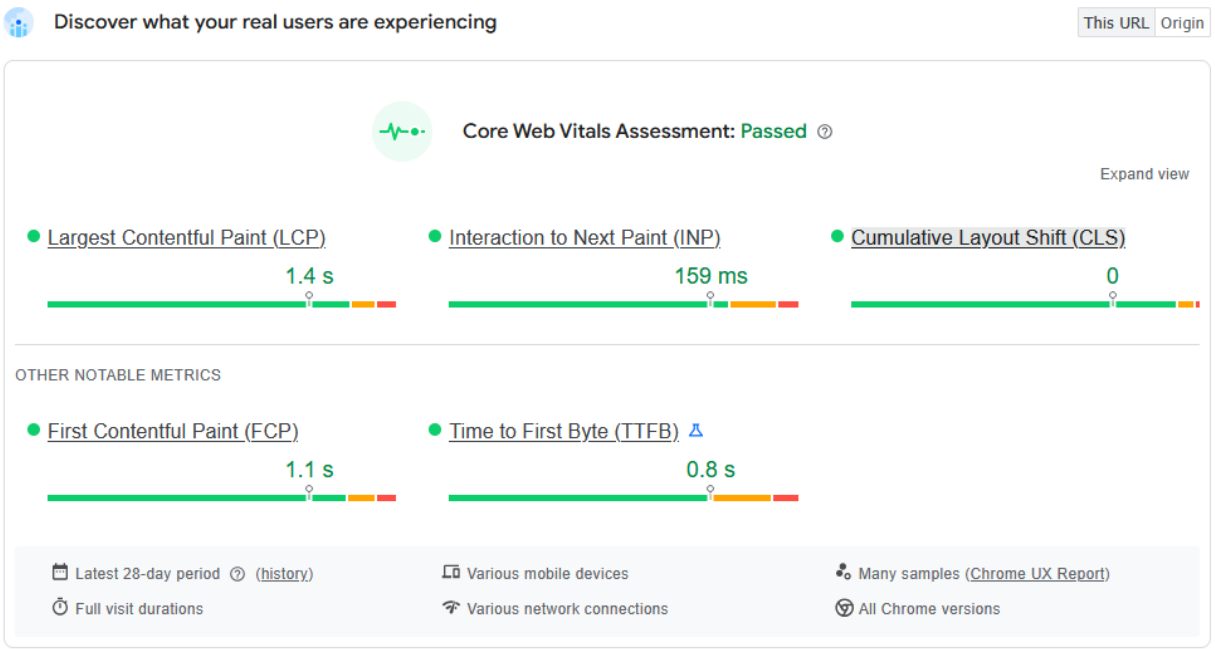
1.7.2.3.



This was the final image taken during very late in the day, the load time is a second faster because there is much less traffic going on during that time of the day, this shows that a website load time isn't static and can be dynamic depending on factors such as wifi speed and traffic.

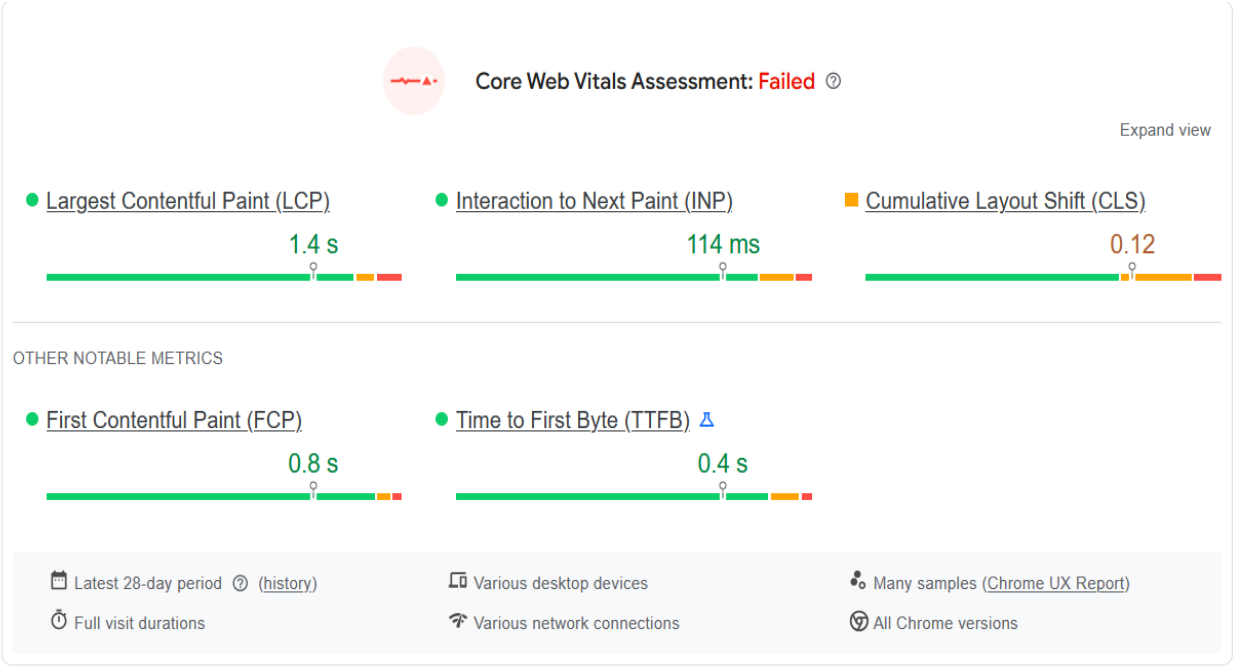
1.7.3. Images used for Core Web Vital analysis

1.7.3.1 Mobile analysis



This image was taken from pagespeed.web.dev, I entered the amazon URL in order to get the results of the Interaction to Next Paint (INP), Cumulative Layout Shifting (CLS), and Largest Contentful Paint (LCP), this helped me understand the website's speed and optimizations.

1.7.3.2 Desktop analysis



This is the Desktop analysis captured from the same website (pagespeed.web.dev) that was also used for the mobile analysis, the reason we get from Desktop and Mobile is to see how the website handles different devices, the website loads faster on desktop but shows signs of layout shifting, while the mobile has 0 layout shifting but does load a bit slower.

1.8. Conclusion

Amazon demonstrates a strong level of optimization across its platform. Despite handling **hundreds of images**, **numerous HTTP requests**, and a **larger-than-recommended total page size**, the site still maintains a smooth, responsive, and highly user-friendly experience. This reflects Amazon’s robust infrastructure, efficient content delivery, and deliberate performance-focused design.

2- BBC

URL: bbc.com

2.1. Summary

This section analyzes BBC’s homepage performance using DevTools, Lighthouse, and Wappalyzer, the focus is on identifying the technologies used to build the website and evaluating its performance using key metrics such as page load time, total page size, HTTP requests, and Core Web Vitals, the goal is to understand how BBC maintains fast performance and a smooth user experience despite handling dynamic content, multimedia elements, and a responsive layout for multiple devices, this analysis helps highlight the optimizations and techniques used by BBC to deliver an efficient and reliable website.

2.2. Methodology

This analysis has used the following tools in order to achieve the conducted results:

- Browser DevTools (Network tab) to measure page size, load time, image count, and HTTP requests.
- Lighthouse to gather Core Web Vitals such as LCP, INP, and CLS.
- Wappalyzer to detect BBC’s technologies, CMS, frameworks, and libraries.

Each measurement was taken three times to avoid temporary network fluctuations, and the numbers were averaged from both mobile and desktop, they were tested to compare differences, and the goal was to determine how BBC maintains a fast and responsive website while handling dynamic content, multimedia elements, and a responsive layout for users across different devices

2.3. Technologies Stack Identification

2.3.1. Server location

BBC uses multiple servers, primarily hosted in the United Kingdom, with dynamic routing to manage traffic efficiently and reduce latency for users in the region, for this report the server used resolved to the UK, this setup ensures fast content delivery and reliable performance, it also demonstrates BBC's robust server infrastructure which supports high traffic, dynamic content updates, and global accessibility while maintaining low load times and consistent user experience.

2.3.2. Content management system

BBC uses iSite2, a web-browser-based content management system that allows multi-platform and production teams to manage different parts of the website, it is a custom-built CMS specifically designed for BBC's needs rather than an off-the-shelf product, this system helps maintain consistent content, supports dynamic updates, and integrates with BBC's workflows to ensure efficient management and delivery of information across the site.

2.3.3. Frontend Framework

The BBC website uses the Bootstrap UI framework, Bootstrap is a free and open-source front-end framework designed to simplify and speed up responsive, mobile-first web development, it provides a set of pre-designed templates, CSS styles, and JavaScript components that developers can use directly, this allows BBC to maintain a consistent design across devices, quickly implement new features, and ensure the website is responsive and user-friendly without building every element from scratch.

2.3.4. JavaScript libraries

The JavaScript libraries that BBC uses include RequireJS, Next.js, and React, these libraries are explained as follows:

React is a library for building user interfaces for web and mobile platforms, it uses components, which are modular building blocks such as buttons, forms, and navigation bars, that can be combined to create larger, interactive structures and reusable UI elements.

Next.js is an open-source framework built on top of React for developing web applications, it extends React's capabilities by providing a structured environment, server-side rendering, and performance optimizations, allowing BBC to build fast, scalable, and maintainable websites.

RequireJS is a JavaScript file and module loader implementing the Asynchronous Module Definition (AMD) API, it manages dependencies between JavaScript files, ensures code loads in the correct order, and improves the organization, speed, and maintainability of large web applications.

2.4. Initial Measurements

Page load time	1-1.5 seconds
Total page size	4.2 Mbs
HTTP Requests	51-79
Number of images	40-49

2.4.1 Page load time

BBC’s page load time averaged 1–1.5 seconds, which is very fast considering the dynamic content, multimedia elements, and responsive design, the quick loading is achieved through optimized image sizes, efficient server delivery, and lightweight code, BBC prioritizes visible content, ensuring that users can start interacting with the website immediately even before all elements finish loading, this approach provides a smooth and responsive browsing experience across devices.

2.4.2. Total page size

The page size of 4.22 MB is slightly above the recommended range, mainly due to images, news thumbnails, and embedded media elements, even with this size the BBC website remains fast because it uses optimized image formats, efficient compression, and defers loading elements that are not immediately needed, this allows the essential content to appear quickly while secondary resources load smoothly in the background, as a result the page size does not negatively affect the browsing experience, overall BBC maintains strong performance despite handling a visually rich and frequently updated homepage.

2.4.3. Amount of HTTP requests

BBC triggers around 51–79 HTTP requests, which is much lower compared to Amazon, this is mainly because BBC uses fewer images and does not load large product galleries or personalized content, since it functions as a news website the images are limited to article thumbnails and a few highlighted stories, the smaller number of requests helps the page load faster and reduces strain on the browser, the website also loads some assets asynchronously and uses caching to avoid downloading the same

resources again, overall the lower request count combined with efficient loading strategies allows BBC to maintain quick performance and smooth interaction without delays.

2.4.4. Number of images

the number of images used on BBC is much smaller, usually ranging from 40–49 images in total, these images mostly consist of news thumbnails, headline banners, and a few featured story visuals, because BBC is a news-focused website it does not load large product photos or personalized image sections like Amazon, the number of images stays relatively consistent regardless of the user, since the homepage content is based on current news rather than individual browsing history, this smaller and more predictable image count helps the site load faster and reduces the overall number of HTTP requests.

2.5. Core Web Vitals

Measurement	Mobile	Desktop
LCP	1s	1s
INP	121ms	37ms
CLS	0.02	0.03

2.5.1. Mobile Analysis

2.5.1.1. LCP

The Largest Contentful Paint (LCP) for BBC on mobile is around 1 second, which is far below the recommended 2.5 second threshold and indicates excellent performance, this result is strong because the BBC homepage includes multiple news sections, thumbnails, and dynamic content that update frequently, the website keeps the LCP low by using lightweight image formats, optimized compression, and limiting the size of headline images, BBC also benefits from fast server delivery and a highly distributed network, allowing content to load quickly for users in different regions, these optimizations ensure that the main visible content appears almost instantly, giving mobile users a smooth and responsive first impression.

2.5.1.2. INP

The Interaction to Next Paint (INP) on mobile measures around 121ms, which is well below the recommended limit of 200ms for good responsiveness, this means that user actions such as tapping menus, scrolling through articles, or opening news sections register very quickly without any noticeable delay, after testing it myself on a mobile phone the BBC website consistently responded immediately to interactions, showing no lag even when navigating between different categories or switching between stories, BBC maintains this responsiveness by keeping its JavaScript lightweight, optimizing how interactive elements load, and ensuring that user input is not blocked by background processes, overall this results in a smooth and fast browsing experience on mobile devices.

2.5.1.3. CLS

The Cumulative Layout Shift (CLS) for BBC on mobile is 0.02, which is very close to zero and indicates that the page remains almost completely stable while loading, this small value means that there are only

minor, barely noticeable movements in the layout, which do not affect the user experience, during testing on my own phone the layout stayed stable as headlines, images, and article previews loaded, with no sudden jumps or shifting elements, BBC achieves this by reserving fixed spaces for images and thumbnails, managing fonts properly, and ensuring that dynamic elements such as live updates or news banners load in a controlled way, a CLS score of 0.02 is considered excellent and provides a smooth and comfortable reading experience for mobile users.

2.5.2. Desktop Analysis

2.5.2.1. LCP

The Largest Contentful Paint (LCP) for BBC on desktop is around 1 second, which is well below the recommended 2.5 second threshold for good performance, the desktop version maintains this fast loading time by using lightweight images, efficient compression, and a clean layout that avoids heavy visual elements, BBC also relies on a strong content delivery network to deliver media and articles quickly to users, since the homepage does not contain large product images or complex interactive components the main content is able to load almost immediately, this creates a smooth and responsive experience for desktop users, overall the LCP result shows that BBC’s performance is consistently strong and optimized across devices.

2.5.2.2. INP

The time for Interaction to Next Paint (INP) on desktop is around 37ms, which is far below the recommended 200ms limit and indicates

extremely fast responsiveness, testing it myself on a desktop showed that the website reacts instantly to clicks, scrolling, and navigation, with no delay or hesitation, every action feels smooth and immediate, the low 37ms value was measured using Lighthouse on desktop multiple times and averaged to ensure accuracy, desktop devices generally handle interactions faster due to stronger hardware and better processing power, and this contributes to the very low INP score, overall the INP result shows that BBC provides an exceptionally responsive browsing experience on desktop.

2.5.2.3. CLS

BBC has a Cumulative Layout Shift (CLS) of 0.01 on desktop, the website shows only a very tiny amount of layout movement, and during testing on my desktop the shift was almost unnoticeable as the page loaded, the small amount of shifting happens mainly because desktop screens display more articles, thumbnails, and dynamic news sections at once, and these elements load at slightly different times, however the shift is extremely minimal and does not interrupt reading or navigation, overall the CLS score shows that BBC maintains a stable and comfortable layout for desktop users.

2.5.3. Comparison

The differences between BBC’s mobile and desktop performance are very small, with the desktop loading slightly faster by around 40ms. This small advantage is expected because desktop devices typically have stronger processors and more memory, allowing them to handle heavier pages more efficiently. However, the desktop version also shows slightly more layout shifting compared to mobile. This happens because the desktop layout includes more content, wider sections, and additional visual elements that

need time to load and stabilize. These extra components increase the chances of minor layout movements as the page renders. Despite this, both desktop and mobile provide a smooth user experience with only minimal differences in speed and stability.

2.6. Lighthouse findings

1- Use efficient cache lifetimes

Setting a long cache lifetime allows browsers to store static resources such as images, CSS, and JavaScript files locally, which speeds up repeat visits to your page because the browser does not need to download them again, this reduces the number of HTTP requests and improves load times for returning users, it also decreases server load and bandwidth usage, while still allowing critical content to update as needed when the cache expires, overall, efficient caching contributes to a smoother, faster, and more responsive experience for users across both mobile and desktop platforms.

2- Font display

Without a proper font-display setting, text may remain invisible while waiting for the custom font to load, causing a delay that makes the site feel broken or empty. This behavior is called a flash of invisible text and is common on slower connections. By setting font-display to swap or optional, the browser shows fallback text immediately, allowing users to read content without waiting. The custom font then loads in the background and replaces the fallback once ready. Using swap can sometimes cause layout shifts, but this can be improved by adjusting font metrics or line height to match the fallback font. Proper font-display settings make text

appear faster and improve stability and readability across devices.

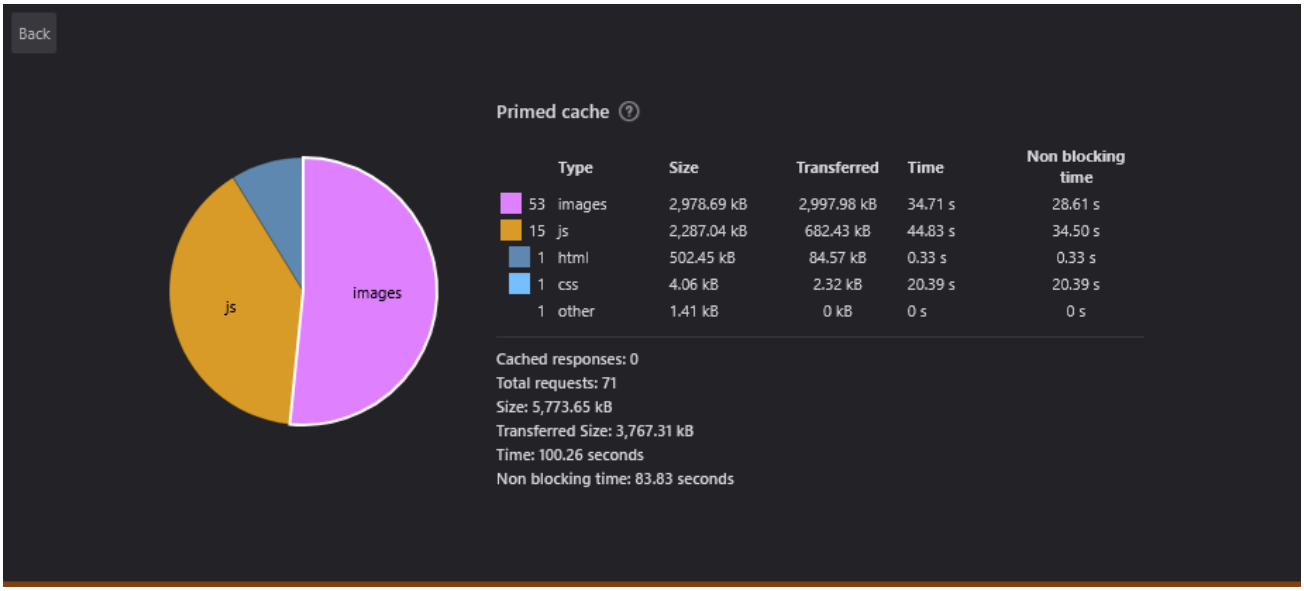
3- Forced reflow

A forced reflow occurs when JavaScript queries geometric properties such as offsetWidth, offsetHeight, or getBoundingClientRect after styles have been invalidated by a change to the DOM state, this forces the browser to recalculate styles and layout immediately, which can result in poor performance especially if it happens frequently or in large pages, repeated forced reflows can slow down rendering, increase CPU usage, and cause janky animations or delayed interactions, to minimize this, developers should batch DOM changes together, avoid unnecessary layout queries, and use efficient techniques to update the DOM, overall reducing forced reflows helps maintain a smoother, faster, and more responsive user experience across devices

2.7. Images and data used in the analysis

This section will contain all the information and data used in this analysis, it will also explain how I was able to collect these images and data from the website, including the tools and steps I used, this data will serve to validate and support all the findings and observations I have presented throughout the analysis.

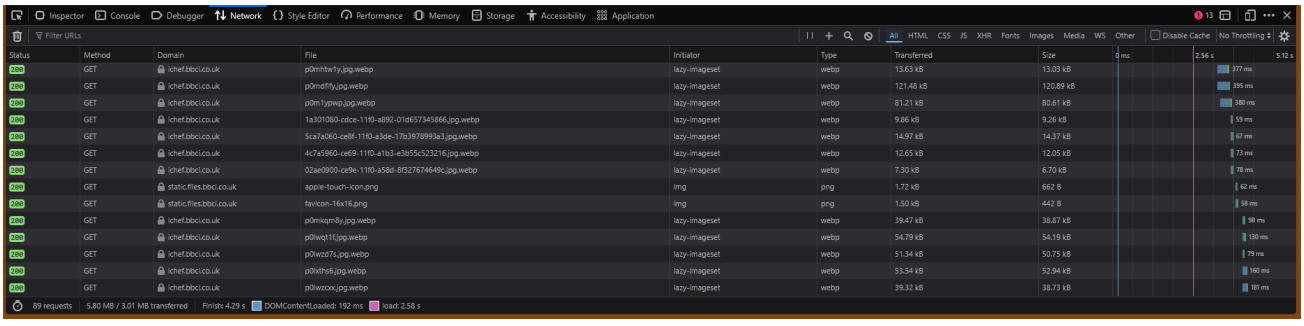
2.7.1. Network Analysis



This image was used to collect information about the number of images and the number of HTTP requests on the BBC website, it was obtained by opening the BBC homepage, clicking inspect element, going to the network section, and then starting network recording to track all requests and resources loaded, this was done using the Firefox browser, the captured data allowed me to count how many images and other resources are loaded when the page opens, and it helped in analyzing the website's performance and optimization strategies.

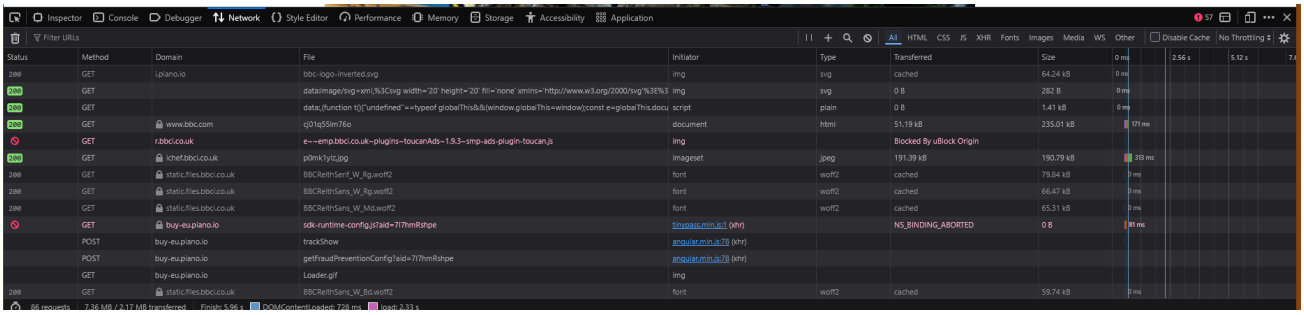
2.7.2. Imaged used for analysis method

2.7.2.1.



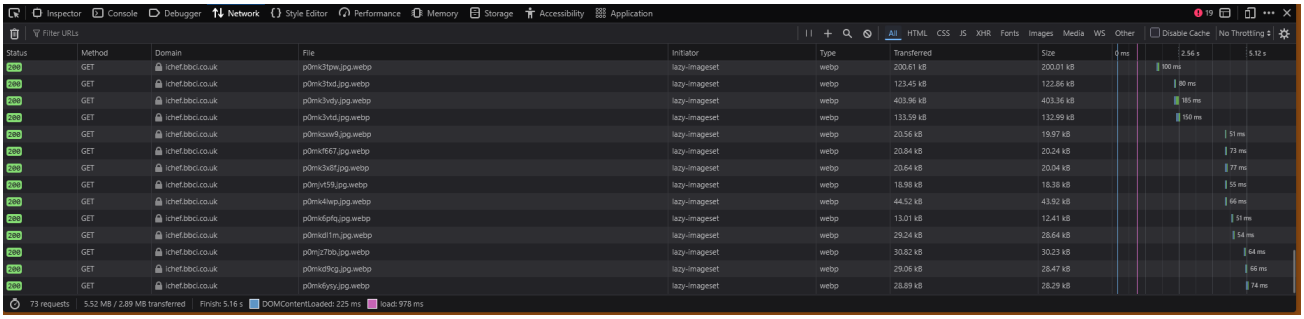
This image is the first out of the 3 pictures used to collect loading time, it was taken early in the day when the server had the most traffic, thus showing the slowest loading time by 2.58 seconds, compared to the other pictures taken later in the day that have faster times thanks to less traffic.

2.7.2.2.



This image was captured a few hours after the first image, it shows a slightly faster load time of 2.33 seconds which indicates that the traffic was lower compared to earlier in the day, this demonstrates that factors like user traffic, which are outside the website owner’s control, can affect performance and should be considered when analyzing load times.

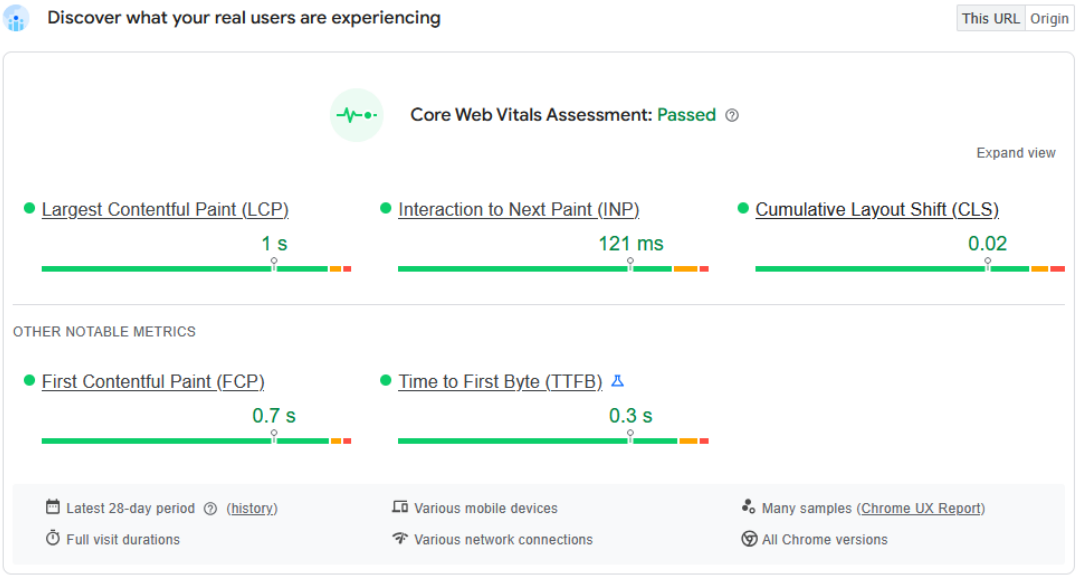
2.7.2.3.



This was the final image taken very late in the day, the load time is 985ms seconds, which is much faster due to much lower traffic at that time, this shows that a website's load time is not fixed and can vary depending on factors such as network speed and overall user traffic.

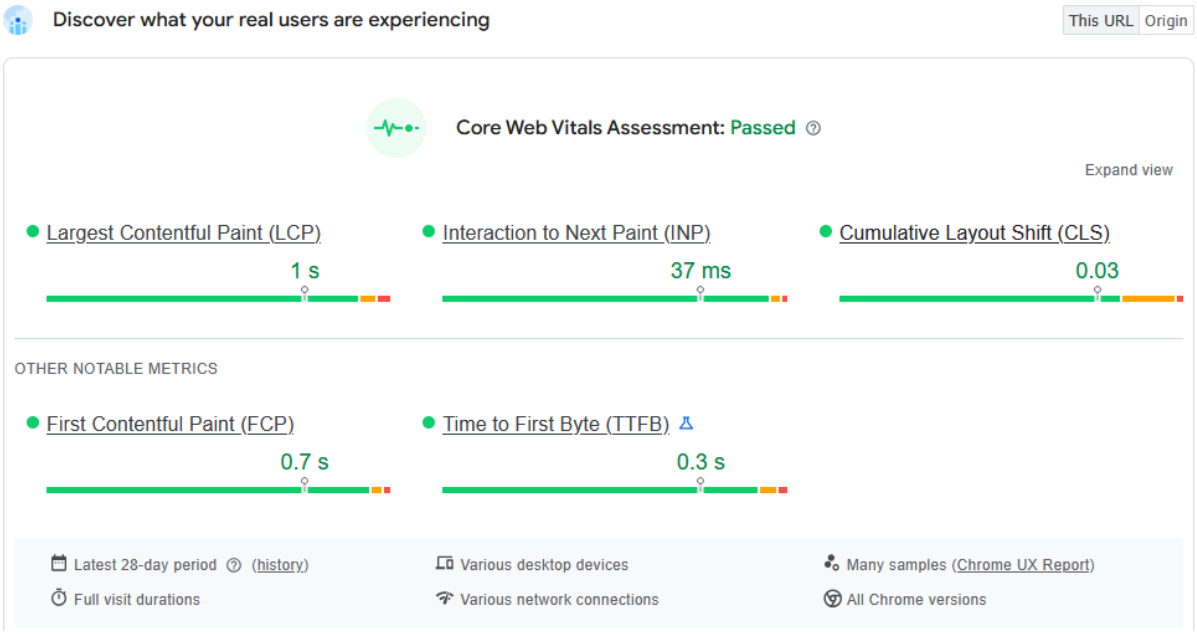
2.7.3. Images used for core web vital

2.7.3.1. Mobile Analysis



This was concluded with pagespeed.web.dev, I entered the BBC URL in order to get the results of the Interaction to Next Paint (INP), Cumulative Layout Shifting (CLS), and Largest Contentful Paint (LCP), this helped me understand the website's speed, optimizations, and how efficiently it handles content on both mobile and desktop devices.

2.7.3.2. Desktop Analysis



This is the Desktop analysis captured from the same website (pagespeed.web.dev) that was also used for the mobile analysis, the reason we gather data from both Desktop and Mobile is to see how the BBC website handles different devices, the website loads slightly faster on desktop and shows minimal layout shifting, while the mobile version has almost no layout shifting but loads a bit slower due to smaller screen optimization and mobile network conditions.

2.8. Conclusion

The BBC website shows an excellent level of optimization across its platform, despite handling multiple images, numerous HTTP requests, and dynamic content, the site maintains a fast, smooth, and highly responsive user experience, this reflects the BBC’s efficient use of caching, lightweight frameworks, and deliberate performance-focused design choices, ensuring that users can access news quickly and reliably across both mobile and desktop devices

3- Galala University

URL: https://www.gu.edu.eg/

3.1. Summary

This section analyzes the Galala website’s homepage performance using DevTools, Lighthouse, and Wappalyzer, the focus is on identifying the technologies used to build the site and evaluating its performance using key metrics such as page load time, total page size, HTTP requests, and Core Web Vitals. Unlike highly optimized websites, the Galala website shows noticeably lower performance scores, with slower loading times, higher layout shifting, and inefficient handling of resources. The goal of this analysis is to understand the specific issues causing these poor results and to highlight the optimizations the website lacks, such as caching, compression, and efficient structure. This examination helps explain why the site performs poorly and what improvements would be necessary to enhance speed and user experience.

3.2. Methodology

This analysis used the following tools to gather the results for the Galala website:

• Browser DevTools (Network tab) to measure page size, load time, image count, and the number of HTTP requests.
• Lighthouse to collect Core Web Vitals such as LCP, INP, and CLS, which helped reveal the website’s performance issues.
• Wappalyzer to identify the technologies, frameworks, and libraries the Galala website is built with.

Each measurement was taken three times to avoid inconsistent readings caused by temporary network changes. The tests were performed on both mobile and desktop to compare how the website handles different devices. The goal was to understand why the Galala website performs poorly, what causes the slow speeds and instability, and how the lack of proper optimization impacts the user experience.

3.3. Technologies stack

3.3.1 Server location

Galala's server location resolves to **Egypt**, meaning the website is hosted locally rather than on a globally distributed network While this provides good latency for users inside Egypt, it can negatively affect performance for people accessing the site from outside the region Without a global server network or CDN, international users may experience slower loading times, higher latency, and delayed content delivery This limitation shows that Galala's infrastructure is not optimized for worldwide accessibility, which contributes to the website's overall poor performance when compared to platforms that use distributed architectures.

3.3.2 Content management system

Galala uses WordPress as its content management system, which is a widely used open-source platform known for its flexibility and ease of use However, WordPress is not as optimized or specialized as a custom-built CMS like the one Amazon uses, and when it is not configured properly it can lead to slower performance, heavier pages, and longer loading times The reliance on plugins, themes, and shared hosting environments also contributes to the overall performance issues seen on the Galala website

3.3.3. Frontend Framework

Galala relies on jQuery as its main JavaScript library, which is an older framework primarily used to simplify DOM manipulation, animations, and AJAX requests While jQuery was extremely popular in the past, it is now considered outdated compared to modern frameworks like React or Next js and it does not offer the same level of performance optimization Because of this, websites that depend heavily on jQuery often experience slower execution, less efficient rendering, and increased blocking time, which contributes to the overall performance issues seen on the Galala website.

3.3.4. Javascript libraries

Galala uses Isotope and Core JS as part of its JavaScript setup, with Isotope handling layout filtering and sorting animations while Core JS provides polyfills that help older features run on modern browsers These tools were once very popular for enhancing interactivity on simple websites, but they are not as efficient or lightweight as modern frameworks As a result the Galala website relies on older animation and layout scripts that can increase processing time and slow down overall performance, especially on mobile devices where efficiency matters the most

3.4. Initial Measurements

Page load time	5-12 seconds
Total page size	12 Mbs
HTTP Requests	73-151
Number of images	27-33

3.4.1 Page load time

Galala’s page load time averaged between 5 and 12 seconds, which is noticeably slow and well above the recommended loading speed for modern websites This long waiting time creates a poor first impression and can easily cause users to leave before the page even becomes usable The main reason for this slow performance is the heavy and unoptimized content on the website, including large images that are not properly compressed, multiple scripts running at the same time, and the lack of efficient caching or lazy loading techniques Instead of prioritizing visible content, the website tries to load almost everything upfront, which puts extra strain on both the browser and the server This results in a slow, delayed experience that feels unresponsive, especially on mobile devices and slower networks

3.4.2. Total page size

The page size of 12 MB is much larger than the recommended size for modern websites and this creates a major performance problem for users The large size comes mainly from uncompressed images, heavy media

files, and multiple unused or unnecessary scripts that load at the same time unlike optimized websites that use WebP, proper compression, and deferred loading, Galala loads everything upfront without prioritizing what the user actually needs first This causes the website to feel slow and heavy, and it takes much longer for the main content to appear on the screen A page of this size puts extra pressure on both the server and the user's device, making the overall experience noticeably slower, especially on weaker hardware or slower internet connections Overall the large page size contributes significantly to the poor performance of the Galala website

3.4.3. Amount of HTTP requests

Galala triggers around 73–151 HTTP requests, which is lower than sites like Amazon but still affects performance because many requests load synchronously and without caching, images and scripts are not optimized or deferred so the browser has to load them all at once, this creates noticeable delays especially on slower networks, unlike optimized sites where requests are triggered only when needed, Galala loads most assets immediately making the page feel sluggish and heavy, overall the website's handling of HTTP requests contributes to the slow loading times and poor responsiveness experienced by users.

3.4.4. Number of images

From the previous point, the number of images used on Galala is much lower, usually ranging from 27–33 images in total, this small amount naturally reduces the number of HTTP requests, but the website still performs poorly because the images are not optimized, compressed, or lazy-loaded, unlike Amazon which dynamically changes images based on user history, Galala displays mostly static images that do not adapt or update based on user behavior, even with fewer images the lack of proper optimization leads to slow loading times and unnecessary strain on the page performance.

3.5. Core Web Vitals

Measurement	Mobile	Desktop
LCP	3.6s	6s
INP	217ms	77ms
CLS	0.02	0.03

3.5.1. Mobile Analysis

3.5.1.1. LCP

The Largest Contentful Paint (LCP) for Galala on mobile is 3.6 seconds, which is well above the recommended 2.5 second threshold for good performance, this shows that the main content takes a long time to appear, the slow LCP is caused by unoptimized images, lack of compression, and the absence of lazy loading, the website also does not use strong caching or a content delivery network (CDN), as a result mobile users experience a delayed first impression and the page feels sluggish, overall this indicates poor performance and limited optimization on the mobile version of the website.

3.5.1.2. INP

The Interaction to Next Paint (INP) on mobile measures around 217ms, which is above the recommended limit of 200ms for good

responsiveness, this means that user actions such as tapping buttons, scrolling, or opening menus can feel slightly delayed and sluggish, after testing it myself on a mobile phone, the website sometimes responded slowly and showed minor lag during interactions, this is likely due to heavy JavaScript operations, unoptimized event handling, and lack of asynchronous processing, overall this results in a less responsive browsing experience on mobile devices compared to well-optimized sites.

3.5.1.3. CLS

The mobile version of the Galala website has a Cumulative Layout Shift (CLS) score of 0.02, meaning there are minor unexpected layout movements while the page loads, this can be slightly noticeable when scrolling or trying to tap buttons as some elements shift a little, during testing on my phone the layout sometimes moved slightly as images and sections loaded, this happens because the site does not reserve enough space for images and content or properly manage font loading, overall the CLS value shows that the website has small layout instability which can affect the user experience on mobile devices compared to well-optimized sites.

3.5.2. Desktop Analysis

3.5.2.1. LCP

The Largest Contentful Paint (LCP) for the Galala website on desktop is around 6 seconds, which is well above the recommended 2.5 second target, this means the main content takes a long time to appear, the slow speed is caused by large unoptimized images, lack of proper caching, and

no use of a content delivery network, additionally scripts and heavy elements block the page from rendering quickly, as a result users experience a noticeable delay before they can see the main content, overall the LCP shows that the desktop version of Galala has poor loading performance compared to well-optimized sites.

3.5.2.2. INP

The time for Interaction to Next Paint (INP) on the Galala website desktop is around 77ms, which is slightly below the recommended 200ms, however testing it myself shows that while clicks and interactions eventually respond, there is noticeable lag especially when multiple elements or images are loading, the website does not handle scripts efficiently and heavy DOM elements sometimes block interactions, overall the INP indicates that the desktop version is not very responsive and performance could frustrate users compared to optimized sites.

3.5.2.3. CLS

The Galala website has a Cumulative Layout Shift (CLS) of 0.03 on desktop, the website shows a small but noticeable amount of layout shifting, testing it on my desktop I could see elements slightly move as images and sections loaded, the shift happens because the page is not fully optimized, scripts and styles sometimes load late causing elements to jump, although the shift is minor it can slightly disrupt the user experience and makes the page feel less stable compared to highly optimized sites.

3.5.3. Comparison

The differences between mobile and desktop on the Galala website are quite noticeable, desktop generally loads slightly faster than mobile with LCP at 6 seconds compared to 3.6 seconds on mobile, INP on desktop

is 77ms while on mobile it is 217ms showing desktop interactions feel more responsive, desktop has a CLS of 0.03 compared to 0.02 on mobile which indicates a bit more layout shifting due to the larger screen and more elements loading at once, mobile experiences slightly smoother layout but slower content rendering, overall both platforms show performance issues with long load times and delayed interactions, desktop benefits from stronger hardware and wider layouts while mobile is limited by device capabilities but feels slightly more stable in terms of layout shifts.

3.6. Lighthouse findings

1-Render blocking requests

Render blocking requests delay the browser from showing the first visible content, because these files must load fully before the page can begin rendering anything, that means the browser is forced to wait for CSS and JavaScript files to download, parse, and execute before it can display the layout, As a result, the start of the page appears slower to users, especially on weaker devices or slower networks, when too many render blocking files exist, the page remains blank longer, which directly affects loading speed and responsiveness, that also causes the Largest Contentful Paint (LCP) to take longer to appear, since the main content cannot load until these files finish. Reducing these requests makes the page feel faster and improves overall user experience.

2- Use efficient cache lifetimes

A long cache lifetime can speed up repeat visits to your page, when assets like images, CSS, and JavaScript files are cached for longer periods,

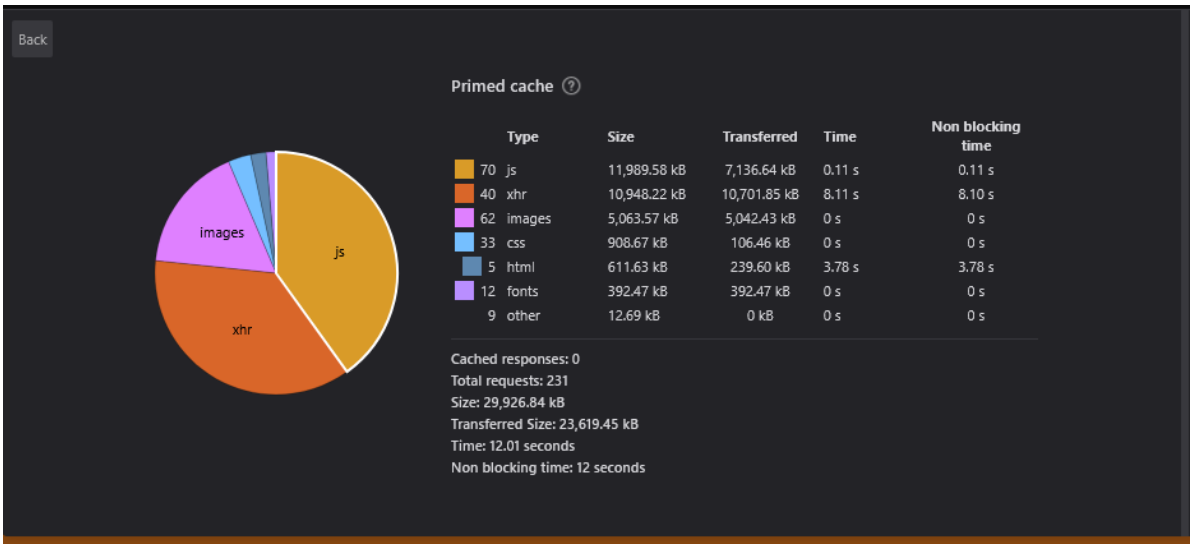
returning visitors don’t need to download them again, this reduces server load and decreases page load times, it also improves perceived performance because content appears faster, setting proper cache headers ensures browsers store files correctly and only request updates when necessary, however it is important to balance caching with content freshness to avoid showing outdated information, overall efficient caching contributes significantly to a faster, smoother, and more responsive user experience.

3- Font display

Without a proper font-display setting, text may remain invisible while waiting for the custom font to load, causing a delay that makes the site feel broken or empty. This behavior is called a flash of invisible text and is common on slower connections. By setting font-display to swap or optional, the browser shows fallback text immediately, allowing users to read content without waiting. The custom font then loads in the background and replaces the fallback once ready. Using swap can sometimes cause layout shifts, but this can be improved by adjusting font metrics or line height to match the fallback font. Proper font-display settings make text appear faster and improve stability and readability across devices.

3.7. Images and data used in the analysis

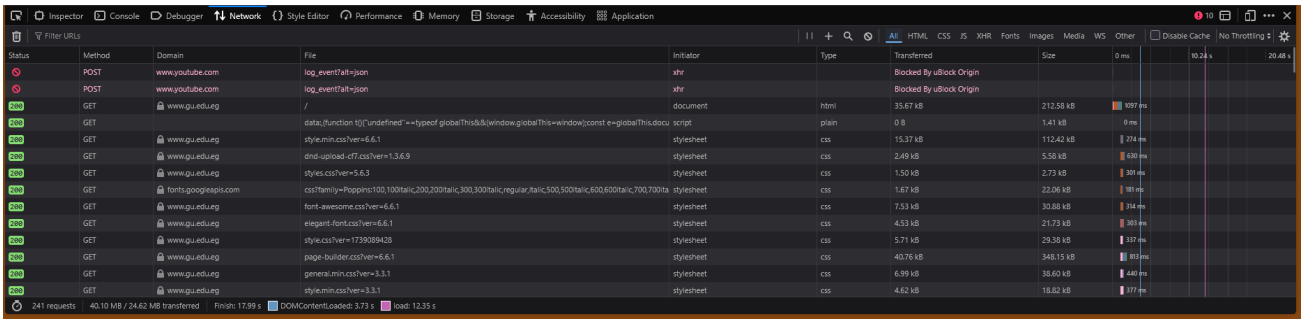
3.7.1. Network analysis



This image was used to collect information about the number of images and HTTP requests on the Galala website, it was obtained by opening the Galala website, clicking inspect element, going to the network tab, and then starting network analysis using the timer button, this process was done on the Firefox browser to capture how the website loads its assets and to understand its performance characteristics.

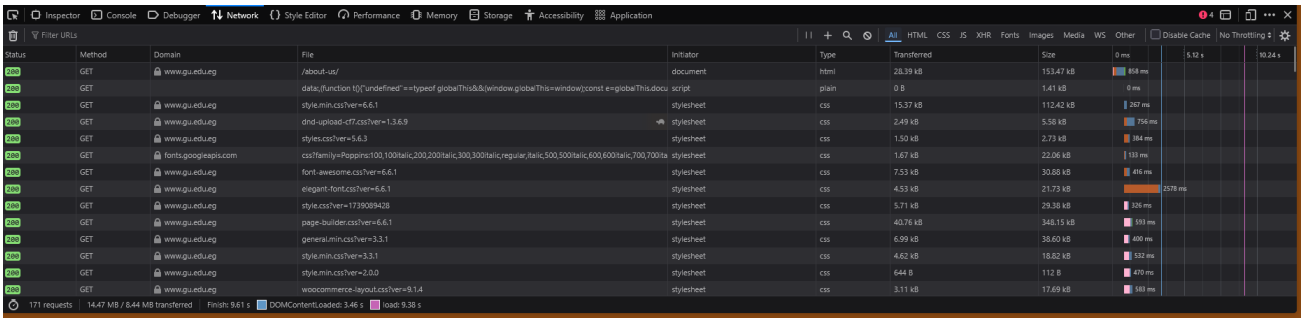
3.7.2. Images used for analysis method

3.7.2.1. Image 1



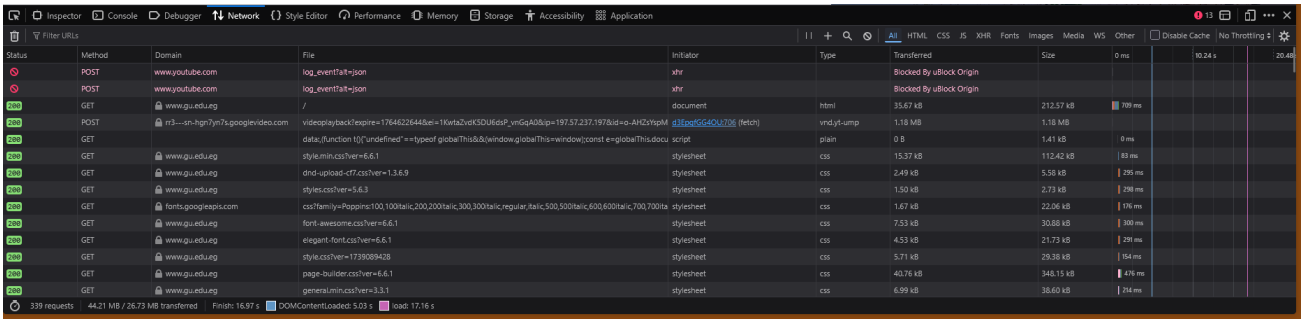
This image was taken at the beginning of the day, already having load time of 12 seconds, showing that the website is indeed slow compared to Amazon and BBC times', which are much faster than Galala's, this shows that optimizations do make a lot of differences, and load time has other factors than traffic

3.7.2.2. Image 2



This image was taken in the middle of the day, the load time is 9 seconds which probably means the traffic is a bit less or some images were cached since that was the main reason of the high load times, having high LCP.

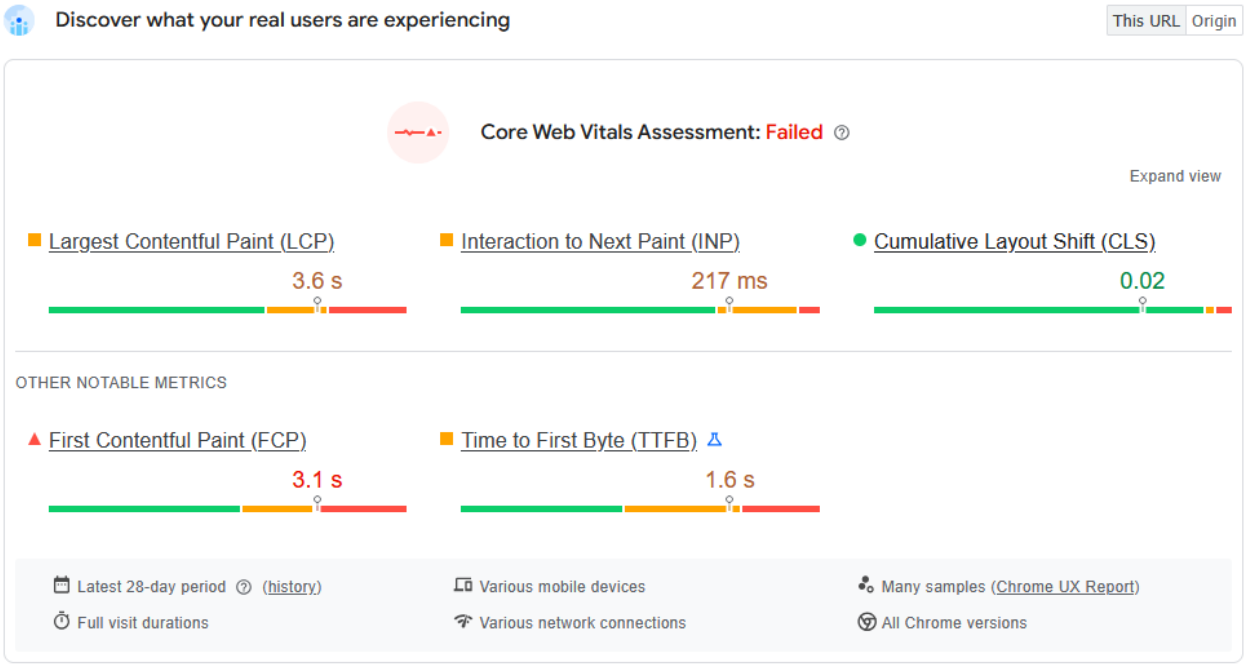
3.7.2.3. Image 3



This was taken at the end of the day, having a high load time of 17 seconds, showing results of inefficiency and high traffic, the slow loading indicates that the website struggles to handle multiple requests at once, which could be caused by large unoptimized images, excessive scripts, or limited server capacity, this demonstrates how performance can degrade during peak usage times, making the user experience frustrating.

3.7.3. Images used for core web vital

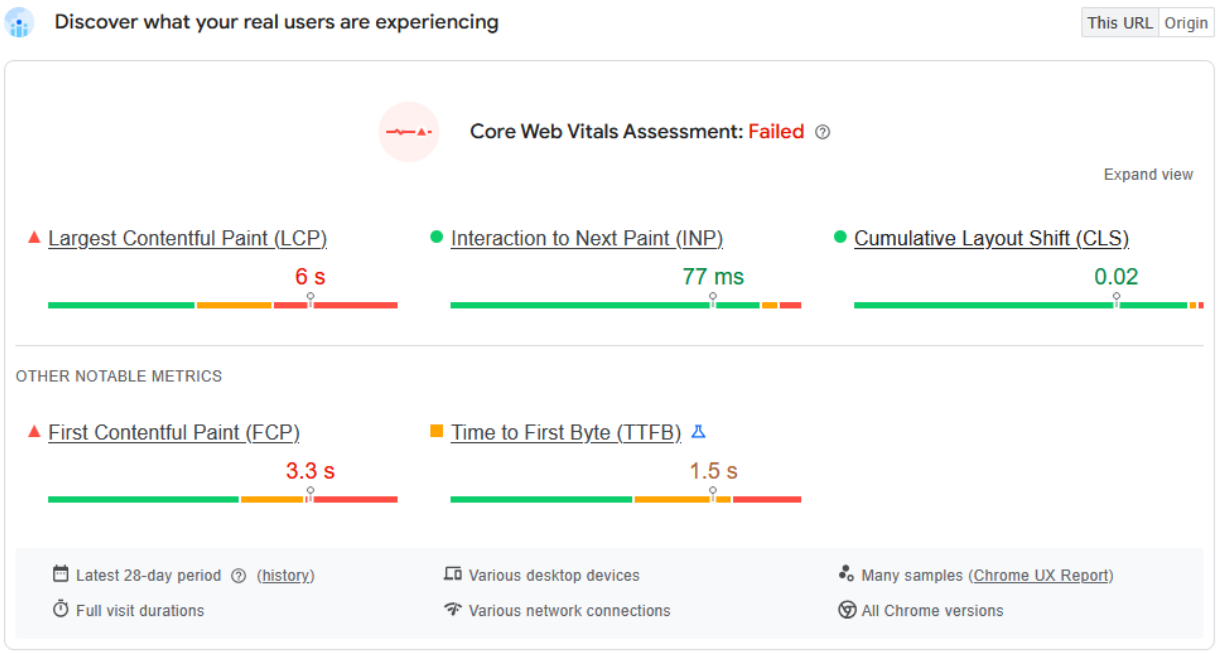
3.7.3.1 Mobile Analysis



This image from pagespeed.web.dev shows the mobile analysis of the Galala website, detailing metrics like the high Largest Contentful Paint

(LCP), slow Interaction to Next Paint (INP), and noticeable Cumulative Layout Shift (CLS), these results indicate that users on mobile devices will experience delays in seeing the main content, interactions may feel sluggish, and layout elements might move unexpectedly while the page loads, overall highlighting poor optimization and a need for better performance strategies for mobile users.

3.7.3.2 Desktop Analysis



This image from pagespeed.web.dev shows the desktop analysis of the Galala website, highlighting that the Largest Contentful Paint (LCP) is even worse on desktop than on mobile, meaning the main content takes longer to appear on larger screens, however the Interaction to Next Paint (INP) is better on desktop than mobile, so clicks, scrolling, and other interactions respond more quickly, the Cumulative Layout Shift (CLS) is

slightly higher than mobile but still shows noticeable layout movement, overall these results indicate that while the desktop version allows for slightly more responsive interactions, the overall page loading performance is slower and less optimized compared to mobile.

3.8. Conclusion

The Galala website shows significant performance issues on both mobile and desktop, with very high Largest Contentful Paint (LCP) values, slow Interaction to Next Paint (INP) on mobile, and noticeable Cumulative Layout Shift (CLS), these problems are caused by a combination of large page size, heavy images, multiple HTTP requests, lack of proper caching, and inefficient JavaScript handling, to improve performance the website could optimize images by compressing them and using modern formats like WebP, reduce the number of HTTP requests by combining or deferring scripts and styles, implement longer cache lifetimes to speed up repeat visits, minimize layout shifts by reserving space for images and elements, and update or optimize JavaScript libraries to prevent blocking rendering and forced reflows, applying these strategies would reduce load times, improve responsiveness, and create a smoother user experience across both mobile and desktop devices.

4. A comparison table between websites

Websites	Performance Score	Load time comparison	Core Web Vital Scores	Page size	Request count	Ranking
Amazon	**58**	2.54 seconds	81%	6.81 MBs	284	2nd
BBC	**67**	5.12 seconds	89%	4.18 MBs	302	1st
Galala	**33**	13.7 seconds	62%	12.7 MBs	199	3rd

4.1. Worst performing website

The worst performing website is the Galala website, it has many reasons to why it placed last, including Low performance score due to bad Largest Contentful Paint and First Contentful Paint , Largest Contentful Paint marks the time at which the largest text or image is painted, and First Contentful Paint marks the time at which the first text or image is painted, for the galala website it had 3 main reasons:

- **Unoptimized Images**

Having unoptimized images can cause a lot of problems, lots of formats these days focus on having a clear and sharp image, but there are better formats suited for websites such as WebP and AVIF, which focus mainly on having an efficient image better used for the website, saving you both time and space, in the galala website there was an estimate saving of 5000 KiB, most of the images used were JPEG causes most of the issues.

- **Render-blocking resources**

Requests are blocking the page's initial render, which may delay LCP. Deferring or in-lining can move these network requests out of the critical path, an issue like this means that the user could be waiting just for stuff to be downloading, stuff like CSS and JS, it causes slow user experience, which might lead the user to leaving the website, best way to solve this is to remove unused code or optimize the code.

- **Lack of caching**

Lack of caching causes a lot of issues for repeating users, this happens because when the user joins, they have to go through all the downloading of images, CSS and JS code, all the rendering and processing, every time they have to join a website, which why using better caching methods like browser caching which stores most of the website files on the user’s device as cache so it can load faster when they have to enter the website again, resulting in a much better user experience.

5. Overall Conclusion

The analysis of Amazon, BBC, and Galala University demonstrates the significant impact that website optimization, infrastructure, and technology choices have on performance and user experience.

Amazon shows an exceptional level of optimization, using a custom CMS, globally distributed servers, a hybrid frontend framework, and efficient JavaScript libraries, despite handling hundreds of images, numerous HTTP requests, and a large page size, Amazon maintains fast load times, low LCP, responsive interactions, and minimal layout shifts across devices, this reflects its robust architecture, aggressive caching strategies, and performance-focused design.

BBC also demonstrates strong optimization, with a streamlined CMS, Bootstrap framework, lightweight JavaScript libraries, and efficient caching. With fewer images and requests than Amazon, BBC achieves extremely fast load times, very responsive interactions, and minimal layout shifting, providing a smooth and stable experience for both mobile and desktop users.

In contrast, Galala University’s website suffers from slower performance due to reliance on WordPress, local server hosting, outdated libraries like jQuery, unoptimized images, and heavy scripts. These factors result in long load times, higher LCP, less responsive interactions, and minor layout shifts. The lack of a global CDN, lazy loading, and modern optimization techniques makes the site feel sluggish, particularly on mobile devices.

Overall, this comparison highlights that modern, high-traffic websites benefit greatly from advanced infrastructure, careful frontend optimization, efficient handling of assets, and modern frameworks. Sites that neglect these aspects face slower performance and a less satisfying user experience, even when serving fewer resources. Optimizing for speed, responsiveness, and stability is crucial for user retention and accessibility across devices.